

Real-AI

The trained MECCG player: vocabulary, the feature specification, the network, how a decision is read out, the weights file, the training and self-play pipeline, evaluation, shipped models, and results

Source of truth: `packages/sim/src/features/`, `packages/sim/src/agents/{bc-agent,from-weights,route-agent}.ts`, `packages/sim/src/cli/{export,export-human,gate}.ts`, `packages/sim/train/`, and `docs/ai-training-system.md` on master, 2026-09-03. Every layout, constant and formula is transcribed from that code; measured numbers cite their run. Companion documents: `heuristics-1.pdf` (the teacher and delegate) and `h2-architecture.pdf`.

1. What Real-AI is
2. Vocabulary
3. The pipeline at a glance
4. Feature specification v1: the global vector
5. Feature specification v1: entity rows
6. Feature specification v1: candidate vectors and the mask
7. The card vocabulary and the action-type list
8. The network
9. The forward pass in TypeScript
10. Reading a decision: argmax, class-mass, temperature
11. Guards: self-test, hashes, cycle guard
12. The weights file
13. Routing: the hybrid
14. Training data: self-play export
15. Training data: recorded human games
16. Behavioural cloning in `train_bc.py`
17. Reinforcement learning: REINFORCE and PPO
18. The self-play loop
19. Evaluation: gates and the value diagnostic
20. Shipped models
21. Results, negative results, and what is known
22. Lobby integration
23. Appendix: file map, CLI reference, flag tables

1. What Real-AI is

Real-AI (registry name `bc`, lobby seat `AI-Real`) is a small neural network that plays MECCG from the same projected view every other agent sees. **It is not a language model.** It is an action-conditioned policy/value network of about 48 000 parameters: given the state and the list of candidate actions, it produces one probability per candidate and one estimate of the game's outcome. The network is trained offline in Python (PyTorch) and played in pure TypeScript, with no runtime dependency, by mirroring the forward pass over weights exported as JSON.

Two kinds of training produce a weights file:

- **Behavioural cloning (BC).** The policy imitates a teacher. **The teacher is recorded human play:** the moves humans made in lobby games, exported as one-hot targets (§15). The code also accepts an AI agent's self-play as teacher (soft targets from its weighted candidate list), and the first models were bootstrapped that way from Heuristics 1 before the human exporter existed, but that is now forbidden as imitation data (see the policy note below). The value head is fitted to the game's final outcome.
- **Self-play reinforcement learning (RL).** Starting from a BC model, the policy plays games at sampling temperature 1 against itself and against a frozen league, and is updated by PPO (or REINFORCE) on the outcomes. A candidate becomes champion only if it beats the previous champion and does not regress against any league member.

Policy (CLAUDE.md, "AI Training Policy"). Only recorded human games may teach the trained AI. No AI seat, Heuristics 1 included, has ever won a game against a human (the corpus stands 107–0 across 21 players), so cloning an AI agent can only reproduce a losing policy. `export-training` output is for opponents, league members, gate baselines and RL rollouts, never for `--mode bc`; and warm-starting a human clone from an AI-cloned base, as the shipped `human-passw-0813` did, is teaching from that AI too. The sections below describe the self-play exporter and the AI-cloned models because they exist and are measured, not because they should be repeated.

The design is deliberately pinned to a versioned *feature specification* and a hashed *card vocabulary*; a weights file that disagrees with the running code on either is refused rather than played badly. What a model is worth is decided only by paired, side-swapped games with confidence intervals; the results section records what has been measured, including the things that did not work.

2. Vocabulary

PlayerView (view)

The per-player projection of the game state, with hidden information redacted. The featurizer consumes only this, never the full `GameState`, so the hidden-information boundary holds by construction.

Candidate

One entry of `view.legalActions: { action, viable, reason? }`. The network scores every candidate, viable or not, so indices line up with the engine's list; only viable ones can be chosen.

Viability mask

A 0/1 vector over candidates, 1 where viable. Applied inside the softmax, so a non-viable candidate gets probability 0.

Forced decision / contested decision

A decision with exactly one viable candidate, versus two or more. About 52% of decisions are forced; they carry outcome signal but no policy gradient.

Feature specification (feature spec)

The exact numeric layout of the global vector, entity rows and candidate vectors, versioned by `FEATURE_SPEC_VERSION` (currently 1). Any change to widths, zone ids, action-type order or scaling must bump it; exports and weights files record it and mismatches are refused.

Global vector

A fixed-width vector (54 columns) summarising the whole view: turn, phase, pile sizes, influence, marshalling points, scores, flags.

Entity row

One 13-column row per visible card in a structured zone (hand, characters, items, allies, attached hazards, sites, permanents, on-guard cards, agents, own site deck).

Zone

The integer id of the structured zone an entity row comes from (`ENTITY_ZONES`, 1–12).

Candidate vector

A 9-column encoding of one candidate action: type index, viable flag, reference count, up to four referenced cards, up to two numeric parameters.

Card vocabulary (vocab)

The mapping from card definition id to an embedding index (1-based, sorted ids; 0 = unknown / hidden / absent). Its FNV-1a hash travels with every dataset and weights file.

Action-type index

The 1-based position of an action's `type` in the append-only `ACTION_TYPES` list (0 = unknown).

Embedding

A learned table row looked up by an integer id (card, action type, zone).

Set encoder / mean pooling

The entity rows are encoded independently and averaged, so the state encoding does not depend on their order or number (a Deep Sets construction).

Torso / state vector

The 64-wide hidden vector combining the encoded global vector and the pooled entities.

Policy head / logits

The per-candidate scorer: state vector and encoded candidate → one logit; the masked softmax over logits is the policy.

Value head / z

A tanh output in $[-1, 1]$ estimating the final outcome from the acting player's side: $z = +1$ win, -1 loss, 0 draw or unknown.

Teacher

Whoever produced the decisions being cloned: `heuristic:sample` self-play by default, or human seats in recorded games.

Soft target / one-hot target

The policy target for one decision: the teacher's normalised candidate weights (soft), or probability 1 on the chosen index (one-hot) when the teacher left no weights.

Trajectory / export

A JSONL file of featurized decisions and per-game results produced by `export-training` or `export-human`.

Holdout

The last fraction of *games* (default 20%), used only to report top-1 accuracy and value MSE.

Top-1 agreement

Fraction of decisions where the network's argmax equals the teacher's choice; reported on contested decisions and overall (forced ones count as correct).

Decode / readout

How one action is chosen from the probability vector: `argmax`, `class-mass`, or `sample` at a temperature. Declared in the weights file because the right choice depends on the teacher.

Temperature

Sampling sharpness: probabilities are raised to $1/T$ before drawing. $T = 1$ samples the policy as is (rollout mode); smaller T approaches the argmax.

Class-mass

Readout that first picks the action *type* holding the most total probability and then the best candidate of that type.

Route

A weights-file declaration handing decisions that offer certain action types to a delegate agent (only `heuristic` is allowed).

Self-test

A real example's inputs and the trainer's outputs embedded in the weights file; replayed at load, with a 2×10^{-4} tolerance, to prove the TypeScript forward pass matches PyTorch.

Warm start (`--init`)

Starting training from an existing weights file; the network shape is taken from it and the decode declaration is inherited.

Behaviour policy / old log-prob / ratio / clip

PPO terms: the probability the rollout policy assigned to the chosen action, and the ratio of the current policy's probability to it, clipped to $[1 - \epsilon, 1 + \epsilon]$.

Advantage

Outcome minus the value head's estimate, $z - V$. In PPO it is fixed once before the update epochs and normalised per rollout file.

Value pre-fit

Epochs of value-head-only training on the new rollouts before advantages are computed.

Entropy bonus / KL early-stop

RL stabilisers: a small reward for keeping the policy spread, and an update that stops when the mean approximate KL from the behaviour policy exceeds a target.

Rollout

A batch of games played by the learning policy at temperature 1, exported as training data.

League

Frozen opponents (by default Heuristics 1) that share the rollouts and gate every promotion.

Champion / candidate / promotion

The recorded best model, the model produced by one iteration, and the act of making the candidate the new champion after it passes every gate.

Gate

A paired-seed, side-swapped match whose pass criterion is the 95% lower bound of the challenger's Elo difference $\geq$ a threshold, with every game completing.

Paired seeds / side swap

Each scheduled seed is played twice with the seats exchanged, cancelling seat, deck and shuffle luck.

Accumulate

Continue learning from the latest candidate whether or not it was promoted; gates only decide what is recorded as champion.

Drift valve / league blowout

Reset the learning line to the champion when a candidate fails a league gate by more than twice the tolerance on the point estimate.

State signature / cycle guard

A coarse fingerprint of a view and a per-game record of actions already taken from it, so an argmax policy cannot ride a costless no-op loop forever.

Determinizer / PUCT / search

The experimental search layer (`search:`) that samples full states consistent with the view and runs a tree search guided by the network. Measured not to beat the bare policy; not shipped to the lobby.

Agent spec

The string that names an agent to the CLIs: `bc:weights.json`, `bc:weights.json@1`, `bc:weights.json@class`, `route:plan-movement+discard-card|bc:w.json|heuristic, search:weights.json@192`.

3. The pipeline at a glance

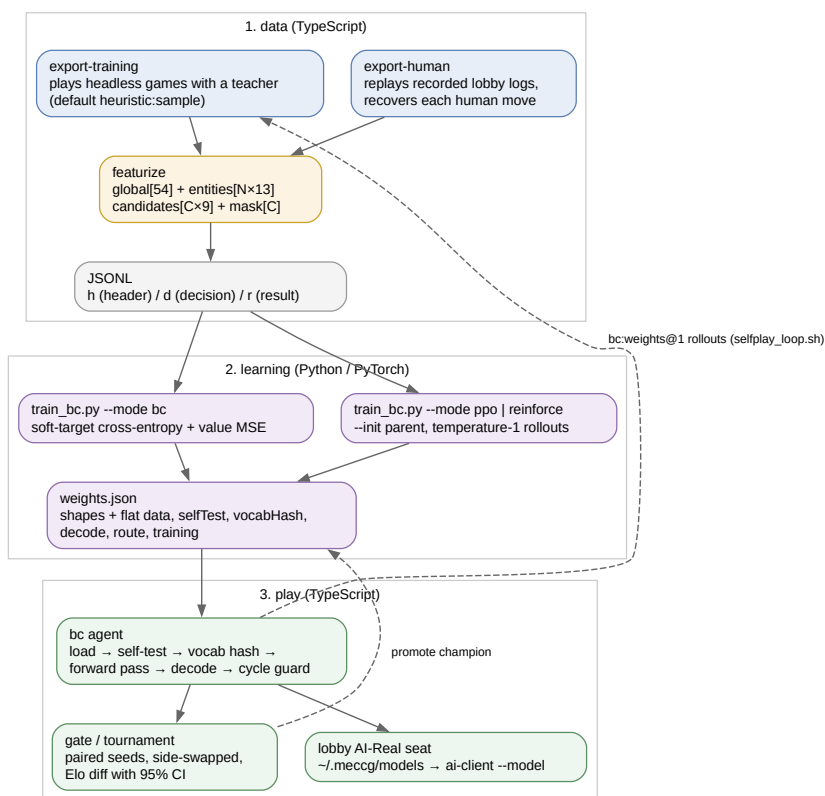


Figure 1. Data is featurized in TypeScript, weights are fitted in Python, and the same TypeScript featurizer plus a mirrored forward pass plays the result.

1. **Data.** `export-training` plays headless games with a teacher and records every decision the teacher made, featurized from the exact context the teacher saw. `export-human` does the same for recorded lobby games, recovering each human move from the game log. Both write the same JSONL format.
2. **Learning.** `train_bc.py` fits the network by behavioural cloning (or updates an existing one by PPO/REINFORCE on rollouts) and writes a weights file that carries everything needed to verify it: feature spec version, vocab hash, dims, decode declaration, a self-test, and a training summary.
3. **Play.** `createBcAgent` loads the file, replays the self-test, checks the vocab hash against the live card pool at the first decision, and then answers each decision with one forward pass and the declared readout. Gates measure it; the lobby serves it.
4. **Loop.** `selfplay_loop.sh` chains rollouts, updates and gates into iterations, promoting when the gates allow.

4. Feature specification v1: the global vector

`featurizeState` emits 54 columns in the order of `GLOBAL_FEATURE_NAMES`. Counts are divided by a constant to keep values roughly in $[0, 1]$; flags are 0/1. The nine phase columns are a one-hot over the `Phase` enum in declaration order.

#	column	value
0	<code>turn</code>	turn number / 100
1–9	<code>phase:setup</code> , <code>phase:untap</code> , <code>phase:organization</code> , <code>phase:long-event</code> , <code>phase:movement-hazard</code> , <code>phase:site</code> , <code>phase:end-of-turn</code> , <code>phase:free-council</code> , <code>phase:game-over</code>	one-hot of the current phase
10	<code>is-active-player</code>	1 if the viewing player is the active player
11	<code>self-index</code>	seat index, 0 or 1 (unscaled)
12–13	<code>self-hand-size</code> , <code>opp-hand-size</code>	/ 10
14–15	<code>self-play-deck-size</code> , <code>opp-play-deck-size</code>	/ 100
16–17	<code>self-discard-size</code> , <code>opp-discard-size</code>	/ 100
18–19	<code>self-sideboard-size</code> , <code>opp-sideboard-size</code>	/ 100
20–23	<code>self-gi-used</code> , <code>self-gi-total</code> , <code>opp-gi-used</code> , <code>opp-gi-total</code>	general influence / 20
24–25	<code>self-stage-points</code> , <code>opp-stage-points</code>	/ 10
26–27	<code>self-deck-exhaustions</code> , <code>opp-deck-exhaustions</code>	count (unscaled)
28–33	<code>self-mp-character</code> , <code>-item</code> , <code>-faction</code> , <code>-ally</code> , <code>-kill</code> , <code>-misc</code>	raw marshalling points by source / 10
34–39	<code>opp-mp-character</code> ... <code>opp-mp-misc</code>	same for the opponent
40–41	<code>self-score</code> , <code>opp-score</code>	tournament score (doubling rule and diversity cap applied) / 10
42–43	<code>combat-active</code> , <code>chain-active</code>	1 if <code>view.combat</code> / <code>view.chain</code> is non-null
44–45	<code>pending-effects</code> , <code>active-constraints</code>	list lengths / 5
46–47	<code>self-companies</code> , <code>opp-companies</code>	/ 5
48–49	<code>self-characters</code> , <code>opp-characters</code>	characters in play / 10
50–51	<code>self-kill-pile</code> , <code>opp-kill-pile</code>	/ 10

The opponent's hand and deck sizes are public in the view, so they are included; their contents are not, and nothing here leaks them.

5. Feature specification v1: entity rows

One row per visible entity, 13 columns in the order of `ENTITY_FEATURE_NAMES`. Integer identity columns (zone, card, bearer, status) are embedded or read as codes on the learning side; the rest are scaled numerics.

#	column	value
0	zone	<code>ENTITY_ZONES</code> id (table below)
1	owner	0 self, 1 opponent
2	card	card-vocabulary index; 0 for a hidden or unknown card
3	status	0 untapped, 1 tapped, 2 inverted (wounded)
4	company	1-based company index within its side (0 = none); self and opponent numbered independently
5	bearer-card	vocab index of the character bearing this item/ally/hazard (0 = none)
6	proWess	definition proWess / 10; for characters in play, effective proWess / 10
7	body	definition body / 10; for characters in play, effective body / 10
8	mind	definition mind / 10
9	influence	definition direct influence / 10; for characters in play, effective DI / 10
10	mp	reads a top-level <code>mp</code> field / 5. Card definitions store marshalling points under <code>marshallingPoints</code> , so this column is 0 for every row in practice (see the note below)
11	corruption	reads a top-level <code>corruption</code> field / 10 (definitions use <code>corruptionPoints</code> , so 0); for characters in play, effective corruption points / 10
12	flag	zone-specific: company site → company has moved this turn; on-guard (own) → revealed; opponent agent → face-down

A quirk to know before changing anything. Columns 10 and 11 read definition fields named `mp` and `corruption`, which card definitions do not carry (they use `marshallingPoints` and `corruptionPoints`). The columns are therefore constant 0 except for the corruption override on character rows. Every shipped model was trained on exactly this layout, so fixing the field names is a feature-spec change: it must bump `FEATURE_SPEC_VERSION` and retrain, not be patched in place.

Zones and row order

Rows follow the view's own iteration order, so the same view always yields the same rows. Character rows use live effective stats; items, allies and attached hazards under a character carry that character's vocab index as bearer.

id	zone	rows emitted
1	selfHand	every card in our hand
2	selfCharacter	our characters, company by company in company order

3	<code>oppCharacter</code>	the opponent's characters, likewise
4	<code>item</code>	each character's items, immediately after the character (both sides)
5	<code>ally</code>	each character's allies
6	<code>attachedHazard</code>	hazards attached to each character
7	<code>companySite</code>	each company's current site (flag = moved)
8	<code>destinationSite</code>	our companies' destination sites; the opponent's only when revealed
9	<code>cardInPlay</code>	permanents in play on both sides
10	<code>onGuard</code>	on-guard cards per company; ours with the revealed flag, the opponent's as the view shows them (hidden ones are card 0)
11	<code>agent</code>	our agents in full (character rows); the opponent's by revealed definition, flag = face-down
12	<code>selfSiteDeck</code>	every site left in our site deck (movement-planning signal)

There is no discard zone; both discard piles enter only as sizes in the global vector. An experimental zone 13 (opponent hazards seen so far this game) exists in `export-human --hazard-memory` only; it widens the entity list past the committed featurizer and was measured not to help (§21).

6. Feature specification v1: candidate vectors and the mask

`featurizeActions` encodes every entry of `view.legalActions` as 9 columns and returns the viability mask alongside. The parameter encoding is generic: the action object is walked with sorted keys (skipping `type`), to a depth of 4; every string that resolves through the view's instance lookup to a card definition becomes a reference, and every finite number becomes a numeric parameter. Identical actions therefore encode identically regardless of how they were constructed.

#	column	value
0	<code>action-type</code>	1-based index in <code>ACTION_TYPES</code> ; 0 = unknown type
1	<code>viable</code>	1 if the engine allows the action
2	<code>ref-count</code>	number of card references found / 5
3-6	<code>ref-card-1 ... ref-card-4</code>	vocab indices of the first four referenced cards (0 when fewer)
7-8	<code>num-1, num-2</code>	the first two numeric parameters / 10 (e.g. a strike's <code>need</code>)

This encoding is what makes the ~200-type, variable-branching action space tractable: the policy scores each candidate against the state rather than choosing from a fixed-width output, so branching is a runtime set size (1 to over 1 300; mean about 8.9) rather than an architecture constant.

7. The card vocabulary and the action-type list

Card vocabulary

`buildCardVocab(cardPool)` sorts the card-pool keys and assigns indices from 1; the FNV-1a 32-bit hash of the sorted ids joined by newlines is the vocabulary's identity. The current pool has 1 683 definitions and hash `6518afc0`. Two rules keep the hash stable against harmless pool changes:

- Synthesised `acts-as-site` companions (Wondrous Maps `td-171` → `td-171-site`) get no index of their own; they alias to the source card, so a company standing on one embeds as the card it is, and certifying such a card does not

renumber everything.

- Index 0 is reserved for unknown: hidden cards (`UNKNOWN_CARD`, `UNKNOWN_SITE`), absent references, and any id not in the pool.

Adding or removing a real card changes the hash and invalidates every weights file; the agent refuses to play on a mismatch rather than embed cards with the wrong rows.

Action types

`ACTION_TYPES` is the ordered list of every `GameAction.type`; a candidate's type column is its position + 1. The list currently has 196 entries: the original 158 sorted names, followed by types appended as cards were certified or rules added.

Indices are permanent and a new type goes at the end, because the list is a serialisation format shared with every checkpoint: certifying `tw-282` once inserted `choose-peek-deck` alphabetically at index 31, shifting 127 of 158 types, and the champion went from 10–9 against Heuristics 1 to 0–18. A completeness test now diffs the list against every discriminant in `packages/shared/src/types/actions-*.ts`, after a drift of 25 types had featurized distinct decisions (a dodge offer, a burglary attempt, a riddling guess) as the unknown row.

A model trained before a type was appended has a shorter type-embedding table (158 or 162 rows plus the unknown row in the shipped files). At inference any type index beyond the table maps to row 0, "unknown", rather than being clamped to whatever type sits last; on warm start the trainer grows the table and initialises the new rows as never-seen types.

8. The network

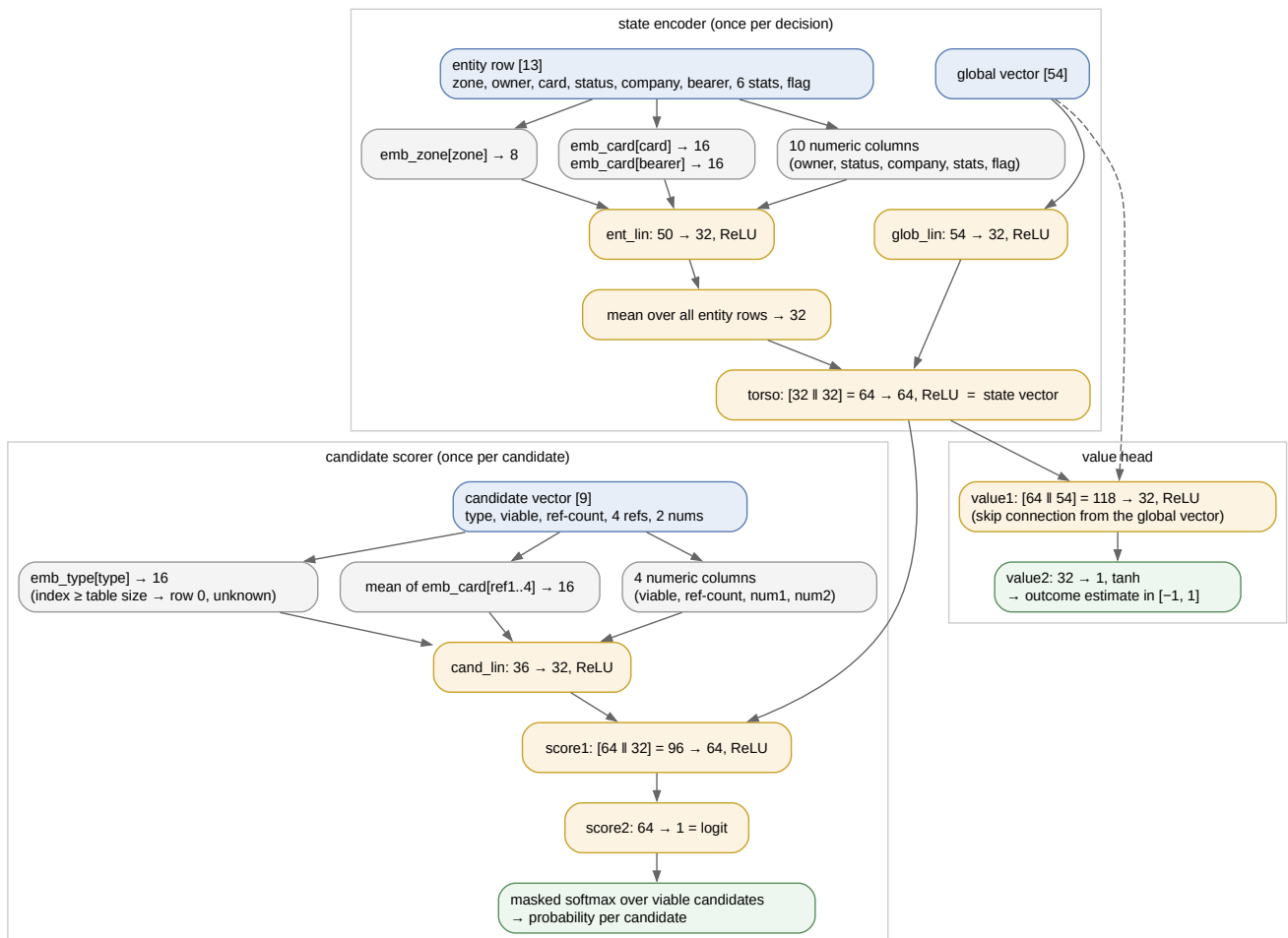


Figure 2. The action-conditioned policy/value network at the `full` preset. Widths are for `full`; `||` is concatenation.

`BcNet` in `train_bc.py` is defined by eight widths, chosen by a preset:

preset	d_card	d_type	d_zone	d_entity	d_global	d_state	d_cand	d_score	parameters (158 types)
large	24	24	12	64	64	128	64	128	≈109 k
full (default, all shipped models)	16	16	8	32	32	64	32	64	48 466
mini (test fixture)	4	4	2	8	8	16	8	16	8 950

Layer by layer at `full`, with the exact tensor shapes the weights file carries (PyTorch `Linear`: weight [out, in], bias [out]):

tensor	shape	params	role
<code>emb_card.weight</code>	[1684, 16]	26 944	card embedding, row 0 = unknown; shared by entity cards, bearers, and candidate references
<code>emb_type.weight</code>	[159, 16]	2 544	action-type embedding (rows = types at training time + 1)
<code>emb_zone.weight</code>	[16, 8]	128	zone embedding; 16 rows leave headroom over the 12 zones
<code>ent_lin</code>	[32, 50] + [32]	1 632	entity encoder over [zone 8 card 16 bearer 16 10 numerics]
<code>glob_lin</code>	[32, 54] + [32]	1 760	global-vector encoder
<code>torso</code>	[64, 64] + [64]	4 160	state vector from [encoded global 32 pooled entities 32]
<code>cand_lin</code>	[32, 36] + [32]	1 184	candidate encoder over [type 16 mean of ref embeddings 16 4 numerics]
<code>score1</code>	[64, 96] + [64]	6 208	scorer hidden layer over [state 64 candidate 32]
<code>score2</code>	[1, 64] + [1]	65	logit
<code>value1</code>	[32, 118] + [32]	3 808	value hidden layer over [state 64 global 54] (skip connection); older files are [32, 64]
<code>value2</code>	[1, 32] + [1]	33	value, then tanh
total		48 466	more than half is the card table; the reasoning part is about 21 500

The entity numerics fed to `ent_lin` are columns 1, 3, 4, 6–12 of the row (owner, status, company, six stats, flag); the candidate numerics fed to `cand_lin` are columns 1, 2, 7, 8 (viable, ref-count, num-1, num-2). Every hidden activation is ReLU.

Why the skip connection. The shared torso is optimised almost entirely by the policy loss (about 1.5 versus 0.2 for the value loss), so a value head reading only the torso scored 0.48 mid-game sign accuracy, worse than the raw tournament-score differential it nominally already had as an input. Since 2026-07-26 the value head also reads the global vector directly. Both runtimes detect which variant a file is by the input width of `value1`, so older files still load.

Capacity is not the lever. Tripling the reasoning capacity (`large`) on the same 2 000 games gave identical top-1 (0.433 vs 0.431), slightly worse holdout value MSE, and heavier overfitting at every data scale (2026-07-27).

9. The forward pass in TypeScript

`bcForward(model, state, actions)` in `bc-agent.ts` mirrors `BcNet.forward` exactly, in float64 over the file's float32-rounded values:

1. **Entities.** For each row: concatenate the zone embedding, the card embedding, the bearer embedding and the ten numerics; apply `ent_lin` and ReLU; accumulate. Divide by `max(rows, 1)`. A view with no rows at all (a player who lost every character with an empty hand) pools to zeros; the collate on the Python side handles the same case.
2. **State.** `glob_lin` + ReLU over the global vector; `torso` + ReLU over the concatenation.

3. **Candidates.** For each candidate: the type embedding (row 0 if the index is past this model's table), the mean of the four reference embeddings, the four numerics; `cand_lin` + ReLU; `score1` + ReLU over [state | candidate]; `score2` → logit.
4. **Policy.** Masked softmax: subtract the maximum viable logit for stability, exponentiate viable logits only, normalise (a zero total is guarded to 1).
5. **Value.** `value1` + ReLU over [state | global] or over the state alone depending on the file, then `value2` and tanh.

Embedding lookups clamp indices into range except for the type table, where an out-of-range index is sent to the unknown row deliberately (§7). The whole pass is a few hundred microseconds to about a millisecond per decision; a policy-only agent cannot wedge a lobby game, which is one reason search is not shipped.

10. Reading a decision: argmax, class-mass, temperature

The probability vector is one thing; the action chosen from it is a separate choice, and the right choice depends on who taught the weights.

readout	rule	suits	spec
argmax	the viable candidate with the highest probability	weights cloned from an agent's <i>soft</i> targets (the teacher scored every candidate, so probability concentrates)	<code>bc:w.json</code> , or file <code>decode.mode: "argmax"</code> , or absent
class-mass	sum probability by action <i>type</i> ; take the type with the most mass; within it, the highest candidate. Ties resolve to the type that appears first in the engine's list	weights cloned from human one-hots	<code>bc:w.json@class</code> , or file <code>decode.mode: "class-mass"</code>
sample at T	draw from $p^{1/T}$ over viable candidates, from the agent's seeded stream	human-cloned weights ($T = 1$); RL rollouts ($T = 1$, so the recorded probabilities are the behaviour distribution)	<code>bc:w.json@1</code> , or file <code>decode: { mode: "sample", temperature }</code>

Why argmax misreads human clones. Argmax assumes every option is one candidate. It is not: `plan-movement` offers a mean of 20 candidates and up to 286, while `pass` is always one. A policy that puts 41% on movement spread over twenty plans and 15% on passing has said "move", and argmax reads it as "pass". A human demonstration is a one-hot over the plan they took, so many similar positions with different chosen plans train a deliberately flat distribution across those plans. Measured on human-trained weights: argmax played movement at 1.6% of the decisions where the human moved and scored 3.5% against Heuristics 1, where sampling the same weights scored 40.6–43%, over 320 paired games each.

Precedence. An explicit option on `createBcAgent` (from the @ suffix) always wins; otherwise the file's declaration decides; a file without one is read at the argmax so that every weights file trained before the field existed keeps the behaviour it was measured with. `class-mass` and a temperature are alternatives, not a pair; asking for both is an error. The agent names itself `bc`, `bc@class` or `bc@T` so replays say which readout played.

Every decision reports the full probability vector over viable candidates as `considered` and the value estimate as its note (`value 0.312`); the explainer renders these as relative preferences, because probabilities are comparable within a decision but not across them.

11. Guards: self-test, hashes, cycle guard

- **Load-time structure.** `loadBcWeights` rejects a file whose `kind` is not `meccg-bc-weights` or `formatVersion` is not 1, whose `featureSpecVersion` differs from the build's, whose `value1` input width is neither the torso width nor torso + global (an architecture this build does not implement), whose decode mode is unknown or whose sampling temperature is not positive, or where any tensor's data length disagrees with its shape.

- **Self-test.** The file's embedded example is run through `bcForward`; the largest deviation across the probability vector and the value must be at most 2×10^{-4} (float32 versus float64 rounding), and a non-finite output counts as infinite deviation so NaN cannot pass a `>` comparison. Failure throws; the agent never plays with drifted arithmetic. This caught an architecture-compatibility bug during development.
- **Vocabulary hash.** At the first decision (and whenever the card pool object changes) the vocabulary is rebuilt from the live pool and its hash compared with the file's; a mismatch throws.
- **Cycle guard.** Some cards make a costless loop legal: I'll Report You (`le-196`) may be returned to hand during any organization phase for nothing, and two argmax agents once burned the full 25 000-decision budget inside turn 1. The agent keeps, per game, the set of action keys (the engine's `actionId`, else `type#index`) already taken from each state signature and excludes them from the readout; if that excludes everything, the unguarded readout applies. `startGame` clears the record, because the signature is coarse and one game's history must never narrow another's choices.
- **First-legal fallback.** If no viable candidate received probability at all, the first legal action is taken with a note saying so.

12. The weights file

A single JSON document, about 550 KB at `full`. The trainer writes it; the agent, the lobby's model picker and `eval_value.py` read it.

field	type	meaning
<code>kind</code>	<code>"meccg-bc-weights"</code>	file identity
<code>formatVersion</code>	1	file-format version
<code>featureSpecVersion</code>	1	feature layout the weights expect; must equal the build's
<code>vocabSize</code> , <code>vocabHash</code>	1683, <code>"6518afc0"</code>	card vocabulary the embeddings index; checked against the live pool
<code>actionTypeCount</code>	158 / 162 / ...	types known at training time (rows of <code>emb_type</code> minus one)
<code>globalWidth</code>	54	global vector width
<code>dims</code>	<code>{ preset, values[8] }</code>	the eight widths of §8
<code>decode</code>	<code>{ mode, temperature? }</code>	how the policy should be read (§10); absent means argmax
<code>route</code>	<code>{ types[], to }</code>	action types delegated, and to whom (§13); absent means the policy answers everything
<code>training</code>	object	provenance: <code>mode</code> (bc/ppo/reinforce), <code>init</code> , <code>actionWeight</code> , <code>balanceAlpha</code> , <code>earlyStop</code> , <code>data</code> (paths, with <code>@seat</code> suffixes), <code>examples</code> , <code>games</code> , <code>epochs</code> , <code>holdoutTop1Contested</code> , <code>holdoutTop10verall</code> , <code>holdoutValueMse</code>
<code>weights</code>	<code>{ name: { shape, data } }</code>	every tensor of <code>BcNet.state_dict()</code> , row-major, values rounded to 7 decimals
<code>selfTest</code>	<code>{ global, entities, candidates, mask, expectedProbs, expectedValue }</code>	one real contested example (from the holdout when there is one) and the trainer's outputs, probabilities rounded to 6 decimals

13. Routing: the hybrid

A cloned policy is not uniformly weak; its competence tracks branching factor, because a one-hot demonstration splits its teaching signal across every candidate it did not name. Measured on 21 808 contested holdout decisions of the human corpus, the deployed weights put 99.7% of their probability on the human's actual `draw-cards` move and 3.3% on their actual `plan-movement`, against about 5% for guessing uniformly among the mean 20 plans offered. The action *mix* was calibrated (it planned movement on 3.3% of decisions, the human's rate), so the policy moved as often as a human and to the wrong places.

`createRouteAgent(types, primary, secondary)` sends a decision to the secondary agent whenever *any* viable candidate carries a routed type; the secondary then answers the whole decision, including the option to pass. Routing on what the engine offers, rather than on a guess at the decision's kind, keeps the rule checkable against the legal-action list. Against Heuristics 1 over 240 paired games each (2026-08-14):

agent	score	paired Elo (95% CI)
plain <code>bc</code> (human-cloned, sampled)	38.3%	-83 [-129, -39]
<code>route(plan-movement)</code>	47.5%	-19 [-60, +21]
<code>route(plan-movement + discard-card)</code>	53.1%	+23 [-20, +68]

Routing the two worst types is worth about 106 Elo (≈ 64 from movement, ≈ 42 from discarding) and reaches parity. Head to head against its parent the hybrid measured +9 [-29, +46], not significant, because the two share weights on about 89% of decisions.

The lobby's Real-AI seat can name a bare file and nothing else, so the composition lives in the file: `route: { types: ["plan-movement", "discard-card"], to: "heuristic" }`. `createAgentFromWeights` builds `bc` (with the file's decode) and wraps it when `route` is present; the delegate is looked up in a local table that contains only `heuristic`, so a dropped-in file can never put a multi-second search into every lobby game. The wrapped agent is named `bc@1+heuristic`. The same composition is reachable from the CLIs as `route:plan-movement+discard-card|bc:w.json@1|heuristic` (the separator is `|` because `,` splits agent lists and `/` appears in every path).

14. Training data: self-play export

```
npm run export-training -w @meccg/sim -- [--games N] [--seed S] [--agents a,b] [--decks d1,d2]
[--out training.jsonl] [--max-decisions N] [--jobs N]
```

Plays headless games and wraps each agent so that every decision is featurized from the exact context the agent saw. Defaults: 10 games from seed 1, `heuristic:sample` on both seats (a sampling agent walks a wider set of trajectories per seed than an `argmax` one), 25 000-decision cap. Under the training policy this exporter produces RL rollouts (the self-play loop calls it with `bc:...@1`) and evaluation material; it is *not* a source of imitation targets, whatever agent it is pointed at. `--jobs N` (or `SIM_JOBS`) fans out over child processes on contiguous seed slices and merges shards in seed order; the result is bit-identical to a serial run. Only the approved challenge decks (a, b, c) may be used for training; `resolveDecks` warns otherwise.

Record formats

line	fields
k: "h" (header, once)	<code>formatVersion</code> 1, <code>featureSpecVersion</code> , <code>vocabSize</code> , <code>vocabHash</code> , <code>actionTypeCount</code> , <code>actionTypes</code> (the names behind the indices, so the trainer can address a type by name), <code>globalWidth</code> 54, <code>entityWidth</code> 13, <code>actionWidth</code> 9, <code>agents</code> , <code>decks</code> , <code>games</code> , <code>baseSeed</code> , <code>maxDecisions</code> , <code>createdAt</code>
k: "d" (decision)	<code>game</code> , <code>seq</code> (0-based within the game), <code>player</code> (seat index 0/1), <code>phase</code> , <code>global</code> [54], <code>entities</code> [N][13], <code>candidates</code> [C][9], <code>mask</code> [C], <code>chosen</code> (index into the candidate list), <code>weights</code> (list of [<code>index</code> , <code>weight</code>] pairs from the teacher's considered list; empty when the teacher reports none)

k: "r"	game, seed, outcome (completed / decision-limit / deadlock / engine-error), winnerIndex
(result, per game)	(null for a draw), winReason, error, finalScores, decisions, turns

Throughput is about 630 examples per second serially, roughly 2 KB per decision. Self-play is also an engine test: any deadlock or engine-error outcome is an engine bug, and this discipline has found 19 distinct defects, mostly offer/validate asymmetries where the legal-action generator offers an action the reducer rejects.

15. Training data: recorded human games

```
npm run export-human -w @meccg/sim -- --dir ~/backup/ai-meccg.com --out human.jsonl [--jobs 8]
```

Every model cloned from self-play inherits its teacher's ceiling; the recorded corpus is the other side of that scoreboard (humans against the AI seats are undefeated). This exporter reads the game server's JSONL logs and writes the identical record format, so human data can be mixed with self-play data or used to fine-tune from it via `--init`.

- **Which seat is the teacher.** Every seat that is not a bot. Bot seats are `AI-Heuristic`, `AI-MC`, `AI-Modular`, `AI-Real`, `Mentor`, plus any name starting `AI-` (the corpus contains 16 games against an `AI-Pseudo` no list mentioned; a bot mistaken for a human would feed the model exactly the policy the data is meant to replace). `--players` restricts to named humans; `--exclude-bots` skips games against named opponents.
- **Recovering the move.** The log records the action that produced each state, so the choice at record N is `records[N+1].action`. It is matched structurally (sorted keys, `actionId` ignored) against the projected view's candidates (*direct*). If that fails, each same-type candidate is applied through the reducer and the resulting state hash compared with the logged next state (*replayed*; exact, because the engine's RNG lives in the state). Anything less certain is dropped and counted as *unmatched*. A move matching a candidate the current engine marks non-viable is refused, since its target index would sit at -10^9 in the masked softmax (5 of 138 000 decisions).
- **Undo moves** (engine-flagged regressive) are dropped: a human backing out of a move is not demonstrating how to play.
- **Unfinished games.** 494 of 631 logs stop mid-game. By default (`--unfinished neutral`) they are exported with `outcome: "completed"`, `winnerIndex: null` and `unfinished: true`: the policy learns from every decision and the value head gets $z = 0$, the honest value of an unknown outcome. `--unfinished skip` keeps only decided games. The winner is read from the log's terminal `game-over` state, not from summaries (only 125 logs have one).
- **Targets.** `weights: []`, always: a human leaves no distribution over the moves not taken, and the trainer falls back to a one-hot on `chosen`.
- **Tally.** The run reports `exported / direct / replayed / duplicate` (several structurally equal candidates encode identically) / `bot / undo / unmatched / non-viable / forced / failed`, so a silently falling attribution rate is visible.

16. Behavioural cloning in `train_bc.py`

```
python3 packages/sim/train/train_bc.py --data train.jsonl [more.jsonl@seat ...] --out
weights.json [--epochs 3] [--batch 64] [--lr 1e-3] [--holdout 0.2] [--seed 0] [--dims full] [-
value-weight 0.5] [--stride N] [--init parent.json] [--decode auto] [--action-weight
pass=0.4] [--balance-alpha  $\alpha$ ] [--contested-only]
```

Loading

- Several files may be given; all headers must agree on feature spec and vocab hash. A `path@0` / `path@1` suffix keeps only that seat's decisions (league rollouts: the opponent's recorded probabilities belong to a different policy and must not enter the gradient). Game ids are namespaced per file (`fileIndex  $\times$   $10^6$  + game`).
- `--stride N` keeps every Nth decision by sequence number and skips the JSON parse for the rest. Decisions within a game share one outcome, so for value learning the binding sample size is games, not decisions; striding buys many more

games per gigabyte. Sizing rule from experience: 1 286 games at stride 4 $\approx$ 25 minutes and 3.4 GB; 3 000 games at stride 2 $\approx$ over an hour and 41 GB.

- Only decisions from games whose result is `completed` are used; `z` is `+1 / -1 / 0` from `winnerIndex` relative to the decision's seat.
- The holdout is the last `--holdout` fraction of games (by id), never a random slice of decisions.

Targets and loss

- Policy target: the teacher's weights, filtered to positive entries and normalised; one-hot on `chosen` when none. Only decisions with two or more viable candidates contribute a policy gradient, but **all** decisions are trained by default, because forced decisions carry outcome signal and the value head starved without them (holdout value MSE sat near 1.0 when they were excluded; `--contested-only` restores that for comparison).
- Loss = policy cross-entropy against the target (masked log-softmax) + `value_weight` (0.5) $\times$ MSE(value, `z`). Optimiser Adam at `--lr`.
- `--action-weight NAME=W,...` scales the policy loss of decisions whose *chosen* action has that type (weighted mean, so the loss scale and the learning rate carry over). Cloning a corpus reproduces its action mix, including parts that are artefacts of how the teacher won; the shipped human models use `pass=0.4`.
- `--balance-alpha  $\alpha$` flattens the chosen-type distribution by inverse frequency: `w = clip((median / count) $^{\alpha}$ , 1/8, 8)`. Without it an unweighted human clone predicted `plan-movement` on 0.1% of decisions against the human's 3.3%.

The decode declaration

Derived from what taught the file unless forced: with `--decode auto`, if at least half the training decisions carry soft weights the file declares `argmax`, otherwise `sample` at `--decode-temperature` (1.0). A fine-tune with `--init` inherits its parent's declaration instead, because self-play rollouts always carry the behaviour policy's soft weights and deriving from them would stamp `argmax` on a human-cloned lineage and gate it at the readout that scores 3.5% instead of 43%.

Warm start

`--init` takes dims and the value-skip variant from the parent, refuses a vocab-hash mismatch, and grows a shorter `emb_type` table to the current type count with the new rows at their default initialisation. Every other shape must match.

Reporting

Each epoch prints policy loss, value loss, and on the holdout: top-1 on contested decisions, top-1 overall (forced decisions count as correct), and value MSE. The final numbers go into the file's `training` block. Reference points: the first 400-game heuristic clone reached 72.1% overall and 42.2% contested against a ceiling of 47.0% (the mean of the teacher's maximum normalised weight, since the teacher samples), so it captured about 92% of what imitation can capture; strength kept improving with data long after top-1 saturated (35.8% $\rightarrow$ 50.0% against the teacher from 40 to 400 games).

17. Reinforcement learning: REINFORCE and PPO

Both modes require `--init` (the policy that generated the rollouts) and rollouts played at temperature 1, so that the recorded candidate weights *are* the behaviour distribution and the chosen action's behaviour log-probability comes free from the export.

`--mode reinforce`

One on-policy epoch: `loss = -(z - V) \cdot \log \pi(\text{chosen}) + \text{value\_weight} \cdot \text{MSE} - \text{entropy} \cdot H(\pi)`, with the advantage detached. Kept as the sanity reference; more than one epoch without importance correction biases the gradient and the trainer warns.

--mode ppo (the loop default)

1. **Value pre-fit** (--value-prefit 1): epochs of training only value1/value2 on the new rollouts, torso and policy frozen. The warm-started head is mis-calibrated on states shaped by unfamiliar opponents, and biased advantages blanket-suppress every action from those games regardless of merit; measured 2026-07-25, heuristic-dominant rollouts lost about 113 Elo on every axis from this alone.
2. **Advantages fixed once**: $A = z - V$ over the whole set, then normalised to zero mean and unit variance *per rollout file* (per opponent family), so games against one opponent can never blanket-suppress what was learned against another. Without normalisation, same-sign batches collapsed the policy into a distinctive draw-spike failure.
3. **Clipped surrogate** over --epochs (4) passes: $\text{ratio} = \pi(\text{chosen}) / \pi_{\text{old}}(\text{chosen})$, $\text{loss} = -\min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 1-\epsilon, 1+\epsilon) \cdot A) + \text{value_weight} \cdot \text{MSE} - \text{entropy} \cdot H$, $\epsilon = \text{--clip } 0.2$, entropy 0.01.
4. **KL early-stop**: after each epoch the mean k3 estimator $(\text{ratio} - 1) - \log \text{ratio}$ is compared with --kl-target (0.02); exceeding it ends the update and marks earlyStop: true in the file.

The loop's learning rate is 3×10^{-5} : 10^{-4} over 4 epochs produced candidate swings from -141 to +47 Elo with repeated collapses. The advantage is the crudest possible (terminal outcome minus value); generalised advantage estimation is the obvious upgrade not yet taken.

18. The self-play loop

packages/sim/train/selfplay_loop.sh <champion.json> <workdir> [iterations], run from anywhere; it changes into packages/sim.

Each iteration, from the current learning line CURRENT:

1. **Self-play rollout**: GAMES games of bc:CURRENT@TEMP against itself.
2. **League rollouts**: for each league member, GAMES_OPP games with the learner in seat 0 (exported as file@0) and GAMES_OPP with the learner in seat 1 (file@1), so only the learner's decisions train.
3. **Update**: train_bc.py --mode MODE --init CURRENT --holdout 0 over all rollout files → candidate-i.json.
4. **Gates**: candidate (read at its file's declared decode) versus the champion at GATE_MIN_ELO, then versus every league member at GATE_LEAGUE_MIN_ELO. League gates always run because they also detect drift.
5. **Outcome**: all gates pass → promoted (champion and learning line move to the candidate). Otherwise with ACCUMULATE=1 the learning line still advances to the candidate unless a league gate's point estimate fell below $2 \times \text{GATE_LEAGUE_MIN_ELO}$ (blowout → reset to the champion). With ACCUMULATE=0 any rejection resets.

variable	default	meaning
LEAGUE	heuristic	comma-separated frozen opponents, e.g. heuristic,bc:/path/frozen.json
MODE	ppo	ppo or reinforce
EPOCHS	4 (ppo) / 1 (reinforce)	update passes over the rollouts
GAMES, GAMES_OPP	60, 30	self-play games per iteration; games per league member per seat
LR, CLIP, KL_TARGET, ENTROPY	3e-5, 0.2, 0.02, 0.01	trainer settings
TEMP	1	rollout sampling temperature; raising it widens exploration (strategies needing several coordinated low-probability moves, such as travelling to a faction and influencing it, were unreachable, and the measured models never scored a faction point), and PPO stays correct because the ratio is taken against recorded probabilities

GATE_PAIRS, GATE_ROUNDS	15, 2	60 games per gate
GATE_MIN_ELO, GATE_LEAGUE_MIN_ELO	0, -25	promotion threshold; no-regression tolerance
SEED0	50000	seed base; each iteration advances by 10 000
ACCUMULATE	1	continue from rejected candidates
SIM_JOBS	1	parallelism inherited by every export and gate call

Why the league exists: pure self-play measurably overfits to its own family; an early candidate gained about +26 Elo over its BC parent while losing about -77 to Heuristics 1, and a later one beat its champion by +47 while regressing -47 against the heuristic, which the league gate correctly rejected. Why accumulation exists: per-iteration candidates were stable (about -25 to +33 Elo, no collapses) but a strict 200-game gate can almost never confirm a true +30 edge, and resetting on every rejection threw the gains away. Rollout files are deleted after each iteration; candidates stay on disk.

19. Evaluation: gates and the value diagnostic

The gate

```
npm run gate -w @meccg/sim -- --challenger <spec> [--champion heuristic] [--pairs 25] [--rounds 8] [--seed 1] [--min-elo -75] [--decks a,b] [--jobs N]
```

Plays `rounds × pairs` seeds, each twice with seats swapped, and passes only when the 95% lower bound of the challenger's score-rate Elo difference is at least `--min-elo` and every game completes; it exits non-zero otherwise, so CI can block a promotion. `--min-elo 0` is the strict "must demonstrably beat the champion" form. Glicko-2 ratings (verified against Glickman's worked example) are reported alongside. The harness records and verifies which git tree it measured, after a period in which failed checkouts produced plausible numbers for the wrong tree (2026-08-16); do not chain pairwise Elo across models, strength here is non-transitive, and measure on at least two deck pairs.

The value diagnostic

```
python3 packages/sim/train/eval_value.py --weights w.json --data holdout.jsonl [--stride N]
```

scores a value head in minutes instead of the hour a search gate costs. Per game-stage bucket (by fraction of the game elapsed: early < 0.25, mid-early < 0.5, mid-late < 0.75, late) and overall it reports MSE against the outcome, sign accuracy on decided games (0.5 = coin flip), and mean |value| (a head that hedges near 0 can have a decent MSE while being useless as a discriminator). Late-game sign accuracy above 0.65 is the threshold it calls "usable as a search evaluator".

20. Shipped models

Files in `~/meccg/models` on the development machine (the lobby lists this directory, newest first). All are `full` dims, feature spec 1, vocab 6518afc0.

field	value
approved-base-0727.json — the base clone	
lineage	behavioural cloning from scratch, 3 epochs, <code>full</code> dims, 158 action types, no decode declaration (argmax)
data	2 473 games / 2.20 M decisions of <code>heuristic:sample</code> self-play on the three approved pairings (a/b, a/c, b/c), stride 2
holdout	top-1 43.4% contested, 71.9% overall; value MSE 1.12

measured	61.3%, +80 [+32, +130] vs Heuristics 1 over 200 games on a/b; +19 [−61, +102] on a/c (2026-07-27). The first model to beat the teacher's point estimate
rebased-g2it6-0727.json — the RL line of record	
lineage	PPO, generation 2 iteration 6 of the loop re-based on approved-base ; 4 epochs; no decode declaration (argmax)
data	160 rollout games: 60 self-play plus league games against two members in both seats (learner's decisions only)
holdout	none (<code>--holdout 0</code>)
measured	its iteration-4 sibling promoted at +63 [+16, +112] vs champion and +51 [+4, +100] vs Heuristics 1 (2026-07-27). A later 60-game gate on the fixed action vocabulary put "trained policy vs heuristic" at 48.8%, −9 [−119, +100] (2026-07-28)
human-passw-0813.json — the human clone	
lineage	behavioural-cloning fine-tune from approved-base , 5 epochs, <code>--action-weight pass=0.4</code> , 162 action types; decode <code>sample @ 1.0</code>
data	537 recorded human games / 138 305 decisions from export-human , unfinished games neutral
holdout	top-1 54.4% contested, 73.0% overall; value MSE 1.17
measured	38.3%, −83 [−129, −39] vs Heuristics 1 over 240 games
human-hybrid-0813.json — the recommended lobby model	
lineage	the same weights as human-passw , plus <code>route: { types: ["plan-movement", "discard-card"], to: "heuristic" }</code>
measured	53.1%, +23 [−20, +68] vs Heuristics 1 over 240 games: parity
packages/sim/test-fixtures/bc-mini-weights.json — test fixture	
lineage	<code>mini</code> dims, 2 games, 2 epochs, 8 950 parameters; regenerate with <code>--dims mini</code> on a 2-game smoke export when the feature spec or network shape changes

The two older files know 158 action types and the two human files 162; the current list has 196, so any of the 34–38 newer types is embedded as "unknown" by these models (§7).

21. Results, negative results, and what is known

What worked

- **Imitation reaches the teacher.** 400 games of heuristic self-play gave a clone at parity with Heuristics 1 (50.0% over 200 games, +0 [−47, +47]); 2 473 approved-deck games gave one ahead of it on a/b (+80 [+32, +130]).
- **Gated RL compounded twice** on the earlier line: `bc-400g` → `runA-it5` (+74) → `gen2-it3` (+72); the champion scored 71.1% (+157 [+122, +194]) against the imitation baseline and 55.1% (+36 [+2, +70]) against Heuristics 1 over 400 games each (2026-07-25). The re-based line promoted again at its first generation (+51 [+4, +100] vs Heuristics 1).
- **The value head was fixed** by training on all decisions, the skip connection, and many more games: mid-game sign accuracy went from 0.49 / 0.48 to 0.84 / 0.76 and overall MSE from 1.09 to 0.56 on 190 held-out games; 1 286 games dropped holdout MSE from about 1.04 to 0.41.
- **Human cloning plus routing** turned a −83 policy into a +23 hybrid against Heuristics 1.

What did not

- **Determinized search does not beat the policy.** PUCT at 192 simulations: 50.5%, +3 [−61, +69] over 100 games. After the value fix and a horizon fix: every configuration's interval still spans zero, 2.7× the budget made it slightly worse, swapping the evaluator for the observable MP spread changed nothing, and 16 worlds hung the process (one 1 200-candidate node costs about 19 000 forward passes). Verdict 2026-07-27: do not ship search; the failure points at strategy fusion in the determinization itself. A flat Monte-Carlo agent with no network, rolling out through the real engine, beats both the policy (+149) and Heuristics 1 (+152 at 8 rollouts, +352 at 16) and scales with budget.
- **The RL recipe plateaued** against a three-member league: two generations (8 and 12 iterations) produced no promotion. Candidates repeatedly gained against their own lineage while losing to the heuristic; the anti-BC gradient is the easier direction.
- **Strength is non-transitive.** A model beat gen2-it3 by +102 and gen2-it3 beat the heuristic by +36, yet that model was −10 against the heuristic on a/b and −78 on i/j, a pair in its own training set. An honest 288-game round-robin on a/b (2026-07-26) put every model within noise of Heuristics 1: gen2-it3 1539 ± 65, heuristic 1521 ± 66, value-skip 1504 ± 65, runA-it5 1436 ± 67.
- **More imitation data does not beat RL, and more capacity does not help.** A 1 520-game multi-deck clone lost −81 to the RL champion; `large` dims matched `full` on top-1 and overfit harder.
- **Blending the score differential into search leaf values is harmful** (2 wins in 12 at weight 0.5); the fitted Heuristics-2 win-probability model as leaf value was inconclusive at 32 games (+22 [−101, +151]).
- **Hazard memory does not help the human clone.** Appending rows for opponent hazards seen this game moved `plan-movement` from 3.3% to 3.6% probability on the human's actual move, still chance. Corpus growth of 18% moved the pure policy from −83 to −47 but the hybrid built on it measured −1 against the deployed hybrid's +23: the extra data improved exactly the decisions routing already delegates.
- **Costless loops** hung argmax agents until the state-signature guard (§11).

Limits

- Trained and evaluated almost entirely on hero-versus-hero challenge decks; minion, Fallen-wizard and Balrog play are untested by training.
- The feature spec is a pure function of the current view: no memory of cards the opponent has spent, no discard-pile contents, no bluffing model for on-guard cards or hidden agents.
- Ratings are relative to a small pool (Heuristics 1, prior champions). The recorded human corpus is undefeated against every AI seat.

22. Lobby integration

- **Model discovery.** `GET /api/models` lists the `.json` files in `~/meccg/models`, newest first, by bare file name. A `play-real-ai` message carries a deck id and one of those names; `resolveModelFile` rejects anything that is not exactly a listed basename (no separators, no `..`), so a lobby message can never smuggle a path.
- **Launch.** The lobby spawns a game server and the headless AI client (`packages/text-client/src/ai-client.ts`) with `--model <path>`; the seat is named `AI-Real`. The client builds the agent with `createAgentFromWeights`, so the file's decode and route declarations apply; without `--model` or `--agent` it plays Heuristics 1. Only approved decks are offered to the AI.
- **Per-decision log.** The client logs the ranked candidate list for every decision with the same renderer the sim's `explain` uses, so a browser "Ask AI" answer and a log line read alike.
- **Ask AI.** The game screen's observer can ask any registry agent, including `bc:<file>`, what it would do at the current position or about the player's own last move; the explanation is derived from the projected view only.
- **Freshness.** A model file goes stale whenever the card pool changes (vocab hash) and silently degrades whenever action types are appended; the weights-free `mc` agent has neither problem, which is why it is the recommended dev-server opponent when cards are moving.

23. Appendix: file map, CLI reference, flag tables

file	contents
<code>packages/sim/src/features/index.ts</code>	<code>FEATURE_SPEC_VERSION</code> and re-exports
<code>packages/sim/src/features/vocab.ts</code>	card vocabulary, FNV-1a hash, acts-as-site aliasing
<code>packages/sim/src/features/action-types.ts</code>	the append-only <code>ACTION_TYPES</code> list and <code>actionTypeIndex</code>
<code>packages/sim/src/features/state.ts</code>	global vector and entity rows
<code>packages/sim/src/features/action.ts</code>	candidate vectors and mask
<code>packages/sim/src/agents/bc-agent.ts</code>	weights loading and validation, forward pass, self-test, readouts, cycle guard
<code>packages/sim/src/agents/from-weights.ts</code>	<code>createAgentFromWeights</code> : decode and route from the file
<code>packages/sim/src/agents/route-agent.ts</code>	type-routed composition
<code>packages/sim/src/agents/search-agent.ts</code> , <code>src/search/*</code>	determinizer and PUCT (experimental, not shipped)
<code>packages/sim/src/cli/export.ts</code> , <code>export-human.ts</code>	the two exporters
<code>packages/sim/src/cli/gate.ts</code> , <code>tournament.ts</code> , <code>src/glicko2.ts</code>	evaluation
<code>packages/sim/train/train_bc.py</code>	network, BC, REINFORCE, PPO, weights export
<code>packages/sim/train/eval_value.py</code>	value-head diagnostic
<code>packages/sim/train/selfplay_loop.sh</code>	the RL loop
<code>packages/sim/train/README.md</code> , <code>docs/ai-training-system.md</code>	workflow notes; the 2026-07-26 system paper with background reading
<code>packages/lobby-server/src/games/models.ts</code> , <code>packages/text-client/src/ai-client.ts</code>	lobby model listing and the headless AI client
<code>packages/sim/src/features.test.ts</code> , <code>bc-agent.test.ts</code> , <code>test-fixtures/bc-mini-weights.json</code>	coverage and completeness tests for the action-type list; forward-pass and agent tests over the mini fixture

CLI cheat sheet

```
# 1. the teacher: human trajectories from the lobby corpus
npm run export-human -w @meccg/sim -- --dir ~/backup/ai-meccg.com --out human.jsonl --jobs 8

# 2. clone from scratch (CPU is fine) with pass down-weighted
python3 packages/sim/train/train_bc.py --data human.jsonl --out human.json --epochs 5 --
action-weight pass=0.4

# 3. self-play rollouts are for RL and evaluation only, never imitation targets
npm run export-training -w @meccg/sim -- --agents bc:human.json@1,bc:human.json@1 --games 60 --jobs 12 --out rollout.jsonl

# 4. play, explain, gate
npm run play -w @meccg/sim -- --agents bc:human.json@1,heuristic --seed 7
npm run explain -w @meccg/sim -- --game <id> --seq <n> --player p1 --agent bc:human.json@class
npm run gate -w @meccg/sim -- --challenger 'route:plan-movement+discard-card|bc:human.json@1|heuristic' --champion heuristic --pairs 60 --rounds 2 --jobs 12

# 5. value diagnostic and the RL loop
python3 packages/sim/train/eval_value.py --weights base.json --data holdout.jsonl --stride 4
LEAGUE="heuristic,bc:frozen.json" SIM_JOBS=12 packages/sim/train/selfplay_loop.sh base.json ~/selfplay/run1 8
```

train_bc.py flags

flag	default	meaning
<code>--data</code>	required, 1+	export files, optional @0/@1 seat filter
<code>--out</code>	required	weights file to write
<code>--epochs</code> , <code>--batch</code> , <code>--lr</code>	3, 64, 1e-3	optimisation
<code>--holdout</code>	0.2	fraction of games held out (by id, last ones)
<code>--seed</code>	0	torch and python seeds
<code>--dims</code>	full	preset; ignored with <code>--init</code>
<code>--value-weight</code>	0.5	weight of the value MSE in the loss
<code>--limit</code> , <code>--stride</code>	0, 1	cap parsed decisions; keep every Nth decision
<code>--mode</code>	bc	bc / reinforce / ppo
<code>--init</code>	—	warm start; required for RL modes
<code>--entropy</code> , <code>--clip</code> , <code>--kl-target</code>	0.01, 0.2, 0.02	RL stabilisers
<code>--value-prefit</code>	1	PPO value-only epochs before advantages
<code>--decode</code> , <code>--decode-temperature</code>	auto, 1.0	readout declaration stamped in the file
<code>--action-weight</code>	—	NAME=W, ... policy-loss multipliers by chosen type
<code>--balance-alpha</code>	0	inverse-frequency exponent over chosen types
<code>--contested-only</code>	off	train only on decisions with ≥ 2 viable candidates

export-human flags

flag	meaning
<code>--dir / --game</code>	corpus root holding <code>logs/games/</code> , or a single log
<code>--out</code>	output (default <code>human-training.jsonl</code>)
<code>--games, --max-decisions</code>	caps
<code>--players, --bots, --exclude-bots</code>	seat selection
<code>--unfinished neutral skip</code>	undecided games as $z = 0$, or dropped
<code>--contested-only</code>	omit forced decisions
<code>--hazard-memory</code>	experimental zone-13 rows; data trains only against itself
<code>--jobs</code>	shard the file list over child processes